See how the `down` method undoes the effect of the `up` method? You can also see a bit of duplication here. In many cases, Rails can detect how to automatically undo a given operation. For example, the opposite of `add_column` is clearly `remove_column`. In such cases, by simply renaming `up` to `change`, you can eliminate the need for a `down`:

```
class AddEmailToOrders < ActiveRecord::Migration
  def change
    add_column :orders, :e_mail, :string
  end
end
```

Now isn't that much cleaner?

Column Types

The third parameter to `add_column` specifies the type of the database column. In the prior example, we specified that the `e_mail` column has a type of `:string`. But what does this mean? Databases typically don't have column types of `:string`.

Remember that Rails tries to make your application independent of the underlying database; you could develop using SQLite 3 and deploy to Postgres if you wanted, for example. But different databases use different names for the types of columns. If you used a SQLite 3 column type in a migration, that migration might not work if applied to a Postgres database. So, Rails migrations insulate you from the underlying database type systems by using logical types. If we're migrating a SQLite 3 database, the `:string` type will create a column of type `varchar(255)`. On Postgres, the same migration adds a column with the type `char varying(255)`.

The types supported by migrations are `:binary`, `:boolean`, `:date`, `:datetime`, `:decimal`, `:float`, `:integer`, `:string`, `:text`, `:time`, and `:timestamp`. The default mappings of these types for the database adapters in Rails are shown in the following tables: